Copenhagen Business School

Natural Language Processing and Text Analytics

Final Project

Name: Sharmin Akther Rima

Student ID: 186412

Detecting Fake Job Advertisements Online – Application of NLP Techniques – DistilBERTa Transformer in specific

---

Total Pages: 15 pages (excluding Cover page, Table of Contents, Executive Summary, References, and Appendix)

Total characters: 33,024 characters (excluding white spaces, from Introduction to Conclusion)

Total words: 4943 words

Submission Date: 10 August, 2026

Table of contents

Executive summary

Fraudulent job postings cost job seekers millions of dollars every year, exploiting fake company profiles and unrealistic offers to commit identity theft and financial fraud. This project goes through a four-tier NLP pipeline on the EMSCAD dataset (17,880 postings with 4.84% fraudulent) to test automated fraud detection with existing text representation quality, using fraud-class F1 and recall — not raw accuracy — as the primary metrics, given the severe class imbalance.

Exploratory Data Analysis included text cleaning and normalization, Structured feature engineering, an 80/20 stratified split, use of class_weight='balanced' to handle class imbalance instead of SMOTE to avoid synthetic text artifacts. Exploratory analysis showed fraud postings have higher company-profile missingness, and given frequent emphasis on buzzwords such as “work home”, “luxury” etc.

Performance is always enhanced with representation richness and correct classifier pairing. Naive Bayes managed only fraud F1 0.44, while Logistic Regression on the same features reached 0.77. Random Forest hit perfect fraud precision (1.00) but missed 42% of scams (recall 0.58). Word2Vec embeddings performed poorly with a linear classifier (fraud F1 0.36) but reached 0.79 with a non-linear MLP, proving that embeddings need the right model to be useful. DistilRoBERTa delivered the best results overall — 99% accuracy, fraud F1 0.85, correctly identified 137 of 173 fraudulent postings, with only 14 false positives and 36 false negatives— by using contextual attention to read full sentences rather than isolated keywords, while remaining light enough to fine-tune on free-tier GPU compute. The DistilRoBERTa fine-tuning time was 748.20 seconds, approximately 12 minutes and 47 seconds for six epochs.

Compared to previous work of Fraud-BERT F1 0.93; BERT/RoBERTa/XLNet baselines at 0.78–0.84, the transformer tier of this project is highly competitive at a fraction of the compute cost, though limited by an aging dataset, token-length truncation, and a single train/test split rather than cross-validation. The results confirm transformer-based NLP is a practical, scalable complement to manual moderation, with future work pointing toward zero-shot LLM comparisons, cross-validation, and eventually edge-computing-based real-time detection to block botnet-driven fraudulent postings before they publish.

1. Introduction

With the rapid growth of the internet, online recruitment platforms such as LinkedIn, Indeed, and Glassdoor gained popularity worldwide for enabling rapid connections between employers and job seekers (Kanimozhi et al., 2026). However, the notable proliferation of fake job postings sometimes cuts the benefits that job platforms provide. Fraudulent job postings make advertisements look attractive by showing fake company growth, promising high salaries for minimal qualifications, and confirming jobs even in the future to keep the resumes. These traps pose severe risks to job seekers, such as identity theft and financial loss (Pillai, 2023). The Internet Crime Complaint Center (IC3) reported that employment scams caused losses exceeding $209 million in the United States alone in 2022. Traditional fraud detection relies on manual moderation and keyword blacklists, approaches that are inadequate in the face of sophisticated natural language- based deception. Machine learning and NLP techniques are playing the greatest role in capturing complex linguistic patterns disguised in the fraudulent job ads. Among many, Multinomial Naïve Bayes has demonstrated strong performance on high-dimensional text data with minimal training overhead, making it particularly well-suited for real-time content moderation scenarios (Kanimozhi et al., 2026). Recurrent Neural Networks and their variants. Recurrent Neural Networks (RNNs) and their variants, such as Long Short-Term Memory (LSTM) networks, have emerged as powerful tools for processing sequential data and capturing the temporal patterns in text. Furthermore, Bidirectional LSTM (Bi-LSTM) networks have demonstrated remarkable performance in various NLP tasks, as they can learn and process contextual information from past and future time steps (Pillai, 2023).

In this project, an NLP-based fraudulent job advertisement detection system is built on the original EMSCAD corpus of 17,880 online job postings, combining text and metadata to classify advertisements as genuine or fake and compare a five-step modeling pipeline having four real tiers:

Tier 1 — TF-IDF + Naive Bayes / Logistic Regression: Establishes bag-of-words and n-gram baselines directly comparing a generative classifier (Naive Bayes) against a discriminative one (Logistic Regression) to test how far simple keyword-frequency patterns can go in flagging fraud, and reporting fraud-class F1/recall rather than accuracy given the ~4.8% class imbalance.

Tier 2 — Random Forest with structured features: Combines TF-IDF text vectors with portal metadata (missing company profile, salary presence, has_company_logo, etc.) to test whether an ensemble tree model captures fraud patterns that plain keyword matching misses.

Tier 3 — Word2Vec embeddings + feed-forward neural network (MLP): Uses pretrained dense word embeddings, averaged per document, visualized via PCA and cosine similarity, feeding a feed-forward network to test whether semantic similarity picks up paraphrased scam language

Tier 4 — DistilRoBERTa fine-tuning: Fine-tunes a pretrained transformer (Byte-Pair-Encoding tokenizer, weighted cross-entropy loss, early stopping, mixed precision) testing whether contextual attention-based representations outperform the static embeddings from Tier 3.

Consolidated evaluation across all four tiers: Merges results into one F1/precision/recall comparison table and confusion matrices, meeting the goal of contrasting simple and advanced NLP approaches on a real, non-synthetic dataset (EMSCAD, via Vidros et al., 2017), while directly addressing the practical, real-world need for scalable and reliable fake-job detection on online recruitment platforms.

Artificial Intelligence and Claude were used to assist in idea generation, structure formulation, coding, and interpretation.

2. Literature Review

The critical and most time-consuming procedures of hiring such as (a) scaffolding job ads, (b) publishing the ads, (c) collecting incoming resumes, (d) managing candidate communication, (e) collaborating on hiring decisions, and (f) reducing the hassle of managing candidates by sorting through thousands of resumes to determine which ones are the best fit for a given position – all are rendered to cloud-based solutions – namely the Applicant Tracking Systems (ATS). One of the core features of an ATS is the streamlined dissemination of new openings to multiple job boards (e.g., Indeed, Monster, CareerBuilder, etc.), social networks (e.g., LinkedIn, Facebook, Twitter, etc.), and email addresses. Manipulation of the functions of the ATS system – specially job-advertisement publishing process – leads to loss of job seekers' privacy, economic damage, and harm to the reputation of the stakeholders. Scam practitioners use 2 types of strategies. The first strategy is "Harvesting Information", where the scammers post fake jobs to collect simple contact details like names, phone numbers, addresses, and, in more extreme cases, educational information and work experience profiles of the job seekers. Afterward, these databases and contextual socioeconomic data-based statistical results are sold to third parties such as aggressive marketeers, communicators for political campaigns, or even to website administrators who plan to send out targeted bulk email messages containing links to generate page views and inbound traffic. The second one is "Identity Theft and Financial Fraud ", which is more sleazier because the scammers pretend to be a legitimate or fictional employer, use ATS to disseminate fake job position contents, and then redirect the users to external methods of communication (i.e., site, email address, or telephone number). They design a fake series of actions, including the dissemination of fake skills tests, scheduling of phony interviews, transmission of congratulatory emails for successful onboarding, etc., to collect sensitive data such as Social Security Numbers, passports, and bank information to misuse and commit financial crimes (Vidros et al., 2017). Reynolds (2021) wrote about nine different types of job search scams, which differ in the type of actions it demands from the job seeker. Job- Hunt, a career advice website, gives an example of 'corporate identity theft', which is fraudulent jobs that claim to be from a real employer (Joyce, 2021).

Traditional fraud detection relies on manual moderation and keyword blacklists, approaches that are inadequate in the face of sophisticated natural language- based deception. Machine learning and NLP offer scalable, automated alternatives capable of capturing complex linguistic patterns that distinguish genuine postings from fraudulent ones (Kanimozhi et al., 2026). Bidirectional LSTM networks, proposed by Aravind Sasidharan Pillai, encode text into dense hidden-state vectors that mix information from every word in a sequence bidirectionally. There's no direct way to trace which specific words or phrases drove a "fraudulent" prediction — the decision emerges from millions of matrix multiplications across gates (forget, input, output) that aren't human-readable. This is a general limitation of most deep neural sequence models, not something unique to the fraud-detection task, but it becomes a real problem here because a job seeker or platform moderator flagged as "fraud" deserves a reason, not just a black-box score. Multinomial Naïve Bayes (MNB) classification with Term Frequency–Inverse Document Frequency (TF-IDF) vectorization (Kanimozhi et al., 2026) applied SMOTE in TF-IDF space that generates synthetic minority samples via k-NN interpolation directly on 10,000-dimension sparse TF-IDF vectors. Interpolating between sparse, high-dimensional text vectors can produce synthetic "documents" that don't correspond to any coherent real text — a well-known criticism of SMOTE on text data.

Naudé et al. (2023) utilized four classes of features: empirical rule set-based features, bag-of-word models, most recent state-of-the-art word embeddings, and transformer models for various machine learning classifiers. Their paper proved Gradient Boosting classifier with a combination of empirical rule-set-based features, parts-of-speech tags, and bag-of-words vectors achieved the best performance with an F1-score of 0.88.

Naudé et al. (2023) extend the EMSCAD dataset beyond binary fraud detection into a 4-class fraud-type taxonomy (real, identity theft, corporate identity theft, multi-level marketing), testing bag-of-words, rule-set,

word2vec, and three transformers (BERT, RoBERTa, XLNet). The paper has notable weaknesses: type-3 postings (only 72 original samples) were upsampled with replacement to 150, meaning half the training data is literal duplicates rather than genuine diversity — a cruder imbalance fix than SMOTE; the entire multi-class analysis rests on just 866 manually- relabeled fraud postings by a single HRM annotator with no reported agreement score; and all three transformers were used largely out-of-the-box with minimal fine-tuning, yet still underperformed a classical Gradient Boosting ensemble (ruleset + POS + bag-of-words, F1 0.883) — BERT scored lowest (F1 0.780), RoBERTa improved on it (0.806) via more robust pretraining, and XLNet performed best of the three (0.839) via permutation-based bidirectional context, notably improving recall on the hardest class, corporate identity theft.

The recent work of Taneja et al. (2025) on transformer based contextual framework called Fraud-BERT. The results of the study conclude that the proposed method is more robust in tackling imbalanced data; it has significantly outperformed existing state-of-the-art studies with an F1 score of 0.93 and 99% accuracy. He argued that most of the machine learning techniques are based on Bag-of-Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF) text representation methods, which are derived from the one-hot representation technique in which the size of the vocabulary depends on the total number of words present in the sentence. These methods represent text with high-dimensional vectors, which are computationally expensive to process. Moreover, these techniques are context-free and cannot preserve the order of words. A few studies have also focused on using deep learning (DL) models such as RNN, GRU, etc., in combination with Word2Vec. Word2Vec is a pre-trained model developed by Google. It is trained on very large corpora of text and represents text with low-dimensional dense vectors. However, Word2Vec can only represent the static context of a word, and DL models need a lot of data for training to give accurate results. In this domain, we generally have to deal with imbalanced data. The disadvantage of traditional ML and DL algorithms is that they cannot handle the data imbalance problem and show bias towards the majority class (Amaar et al., 2022).

The Fraud-BERT paper (Taneja et al., 2025) exhibits several methodological limitations when evaluated against standard NLP research practices. The model was fine-tuned for only 2 epochs with a fixed learning rate of 1e-5 and a batch size of 6, with no reported validation curves or justification beyond a claim that additional epochs caused overfitting. The comparison baselines are also asymmetrically configured — the Bi-LSTM baseline uses only two layers of 60 units each, an embedding size of 300 trained from scratch (no pretrained GloVe or Word2Vec vectors), and just 2 training epochs, making it a considerably weaker benchmark than a properly tuned recurrent model would be. Additionally, the study relies exclusively on BERT-base (110M parameters), despite the authors themselves acknowledging that BERT carries high computational overhead, cannot process sequences beyond 512 tokens without truncating longer job postings, and lacks interpretability.

Among the projected model tiers (Naive Bayes, Logistic Regression, Random Forest, Word2Vec+MLP, and DistilRoBERTa), DistilRoBERTa carries roughly 40% of BERT's parameters, fine-tunes on a free Colab GPU in minutes, and still gives a genuine transformer tier to discuss — directly solving the deployment-cost problem. Instead of SMOTE's synthetic text-vector interpolation or upsampling - with-duplicates, *class_weight='balanced'* uniformly across tiers — a single parameter that solves the same problem without inventing any data, and prioritizes fraud-class recall as your primary metric given the ~4.8% imbalance. The DistilRoBERTa model was fine-tuned for **6 epochs** on an NVIDIA Tesla T4 GPU, with total training taking approximately **748.2 seconds (12.47 minutes)**. It achieved its strongest validation performance at epoch 5, reaching 98.46% accuracy and an F1-score of 0.828, indicating effective detection of fraudulent job-posting patterns. Here, a comparison chart is shown with related works on the EMSCAD dataset.

Ref. & Authors (Year)	Models Evaluated	Highest Performing Model (with results)	Key Contribution / Focus
Marcel Naudé et al. (2022)	Logistic Regression (LR), Stochastic Gradient Descent (SGD), k-Nearest Neighbors (KNN), Decision Tree (CART), Support Vector Machine (SVM), Random Forest (RF), AdaBoost (AB), and Gradient Boosting (GB)	Gradient Boosting (GB) with F1 0.88	Word embeddings and transformer-based features outperformed the handcrafted rule-set-based features class.
Aravind Sasidharan Pillai (2023)	Ensembled models (Random Forest, LightGBM, and Gradient Boosting Machine (GBM), Bidirectional LSTM	Bidirectional LSTM with 98.71% accuracy and F1 0.86	Deep learning techniques for fraudulent job detection – considering both numeric and text features.
Akram et al. (2024)	BERT, RoBERTa paired with SMOBD SMOTE	BERT + SMOBD WITH 90% balanced accuracy, 97.03% overall accuracy	Utilized contextual encoders to detect subtle linguistic anomalies and density-based oversampling to generate synthetic samples strictly within minority semantic boundaries.
Khushboo Taneja et al. (2025)	Fraud- BERT, LR, SVM, NB, RF, Bi - LSTM	Fraud- BERT with 99% accuracy, 0.93 F1 score.	Leverages contextual transfer learning to detect recruitment fraud, overcoming severe data imbalance without needing extra pre-processing or resampling.
Jonathan et al. (2025)	BERT, RoBERTa, DeBERTa, ELECTRA, XGBoost, RF, LR	BERT WITH 98.88% accuracy, F1 score 0.8802	Finding and fixing synthetic data problem.
Rahman and Ahmed (2026)	XGBoost, RF, RoBERTa	XGBoost with 99.42 % accuracy and 0.99 F1 score	SMOTE to handle severe class imbalance.

Kishore et al. (2026)	BERT embedding and SMOTE - SMOBD	Balanced Accuracy 80.9%, overall accuracy 86.6%	SMOBD variant of SMOTE enhances minority class representation to prevent classification bias.
Proposed model	TF-IDF + NB, TF-IDF + LR, RF with structured features, Word2Vec embeddings + feed-forward neural network (MLP), DistilRoBERTa fine-tuning	DistilRoBERTa (accuracy 99%, f1 score – 0.85)	understanding the context of the full job description

Table 1: Comparison of models from different authors on EMSCAD dataset

3. Research Methodology

3.1 Proposed Framework

This paper suggests a binary fraud classifier for job postings (target: fraudulent, 0 = real, 1 = fake) on the EMSCAD dataset (N=17,880) to identify fake job advertisements. The pipeline will tackle class imbalance (~4.8% fraud) using class_weight='balanced' rather than SMOTE to avoid generating synthetic text artifacts, and the models are judged primarily on fraud-class F1-score and recall rather than raw accuracy, since accuracy is misleading on this imbalanced dataset. The methodology consists of the following phases:

1. Data Acquisition and Environment Description
2. Exploratory Data Analysis
3. Pre- processing and Feature Engineering
4. Comparative Classification with Machine Learning, Deep Learning, and Transformer-based architectures

3.2 Dataset Description

The analysis will be done on a publicly available dataset, (EMSCAD), Employment Scam Aegean Dataset, which was available on Kaggle. The dataset consists of 17,881 job postings with 18 columns having both text fields (title, company profile, description, requirements, benefits) and structured/ boolean metadata (has_company_logo, telecommuting, has questions, employment type, required experience/education), with the binary target fraudulent marking 866 postings (4.84%) as fake versus 17,014 (95.16%) as real, confirming the severe class imbalance central to this project.

3.3 Environment Description

The study is conducted on Google Colab using a T4 GPU environment. All steps were executed and documented within the same cloud-based environment.

```
torch: 2.11.0+cu128
transformers: 5.13.1
scikit-learn: 1.6.1
GPU available: True
Device: Tesla T4
```

3.4 Exploratory Data Analysis

3.4.1 Missingness by class

The missingness data suggest that fraudulent job postings leave out enriched company profile information (67.8%) more than real postings, making this a useful indicator of fraudulent job scams. On the contrary, salary range, benefits, requirements, and department are missed out at almost similar rates in both categories. They do not contribute to fraudulent ad detection.

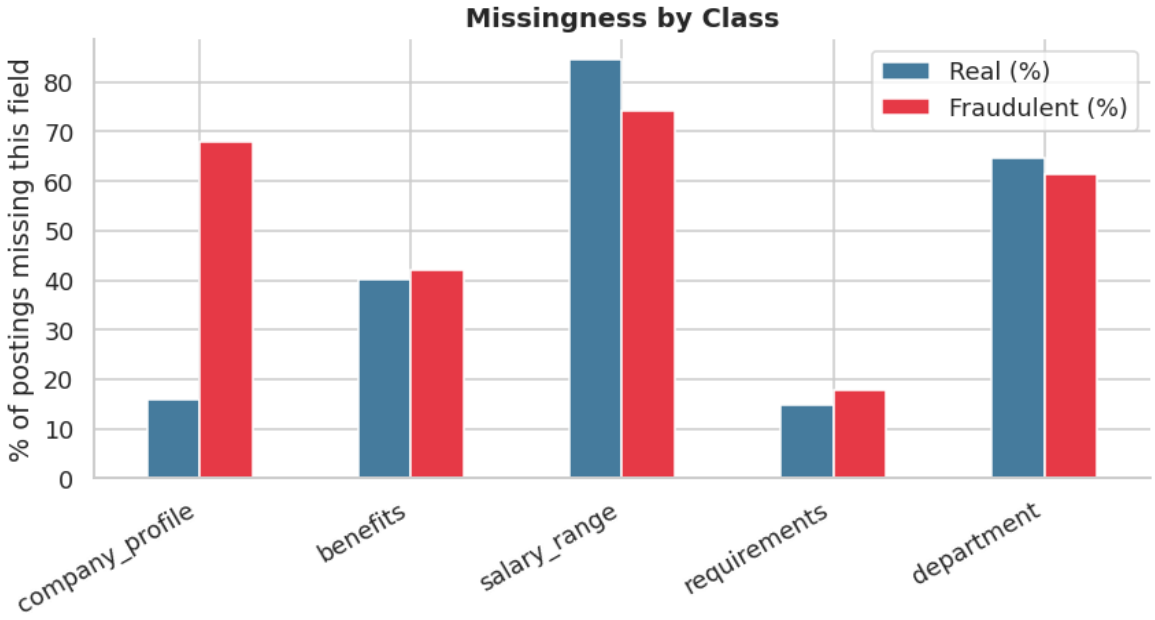


Figure 1: Missingness of data by class on different indicators

3.4.2 Text length by class

Real job postings tend to have slightly longer descriptions (mean length of 171 words and standard deviation of 122.6), whereas fraudulent ads typically use fewer words and provide less detailed descriptions (mean length of 158.7 words and standard deviation of 136.6).

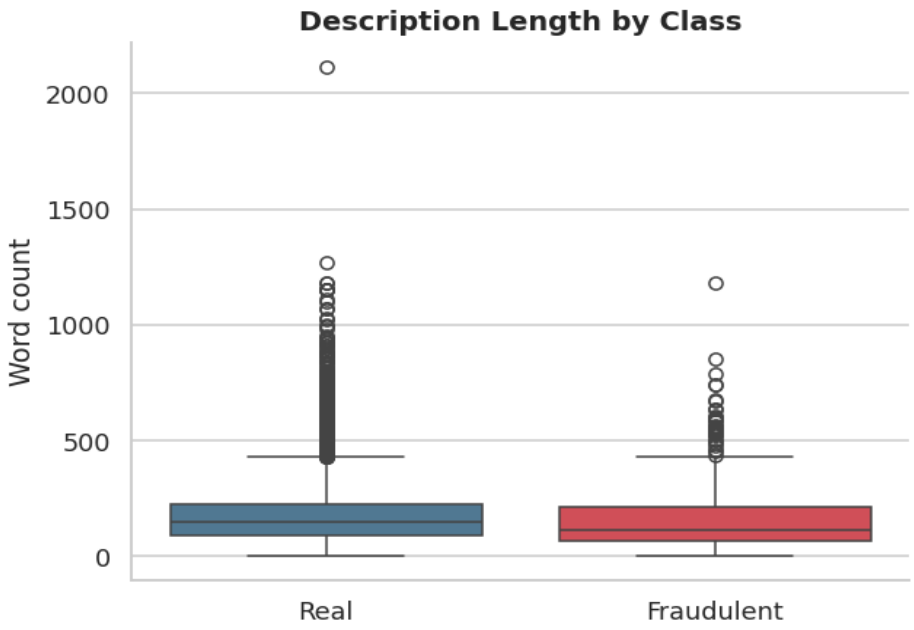


Figure 2: Description of job advertisements by class

3.4.3 Top bigrams by class

Real postings use phrases such as customer service, social media, join team, ideal candidate, etc., more frequently (about 240, 190, 130, 100 times consecutively), highlighting more emphasis on qualified candidate fit. Whereas, fraudulent job ads rely more on industry buzzwords, enticing but vague terms that look attractive such as oil gas, data entry, gas industry, work home, luxury etc.

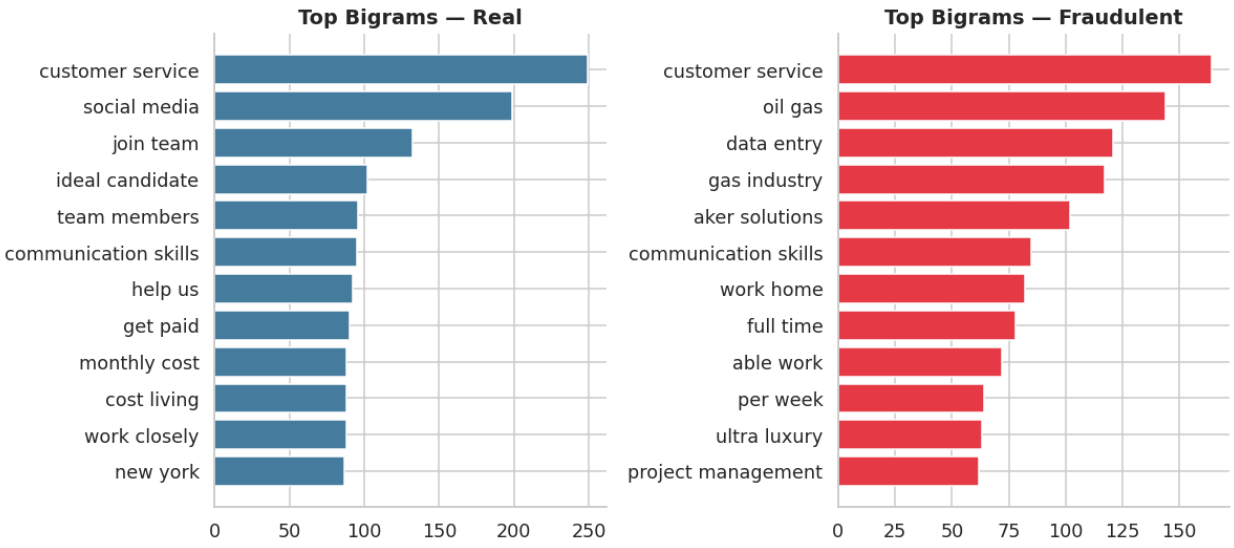


Figure 3: Top Bigrams – the most common words – used in job postings by class

3.5 Data Pre-processing and Feature Engineering

3.5.1 Text cleaning and normalization: Raw job descriptions were converted into a clean_text field by lowercasing all words, removing punctuation, URLs, and HTML entities, and simplifying tokens so that only meaningful content (roles, skills, responsibilities, company info) remains for modeling.

3.5.2 Structured feature engineering: Transformation of job attributes and missingness indicators into binary columns (e.g., telecommuting, has_company_logo, has_questions, missing_company_profile, missing_benefits, missing_salary) was done so that the model can learn patterns from the presence/absence of features and fields.

3.5.3 Stratified train–test split under class imbalance: The data was split into about 80% train (14,304 samples) and 20% test (3,576 samples) using stratify=y, ensuring that the fraud rate (~4.84% of postings) is the same in both sets, which makes evaluation metrics reliable for this imbalanced fraud-detection task.

3.5.4 Handling class imbalance: To handle class imbalance, every classifier is trained with class_weight='balanced', which automatically gives higher weight to the minority fraud class and lower weight to the majority real class. This encourages the model not to default to predicting only real postings and achieves the same goal as oversampling, but without generating any synthetic text data (avoiding potential issues with SMOTE on sparse TF-IDF features).

3.6 classification models

3.6.1 Baseline Tier 1: Bag-of-Words (Naive Bayes + Logistic Regression)

Tier 1 represents each job posting as a TF-IDF bag-of-words vector (unigrams and bigrams, up to 10,000 features) and trains two classifiers on it. Multinomial Naive Bayes is a generative probabilistic model that assumes words are drawn independently from class-specific distributions and applies Bayes' rule to assign the posting to the most likely class. Logistic Regression is its discriminative counterpart: it learns a weighted sum of the same TF-IDF features, squashes it through a sigmoid function to produce a probability between 0 and 1, and labels the posting as Real or Fraudulent based on a chosen threshold.

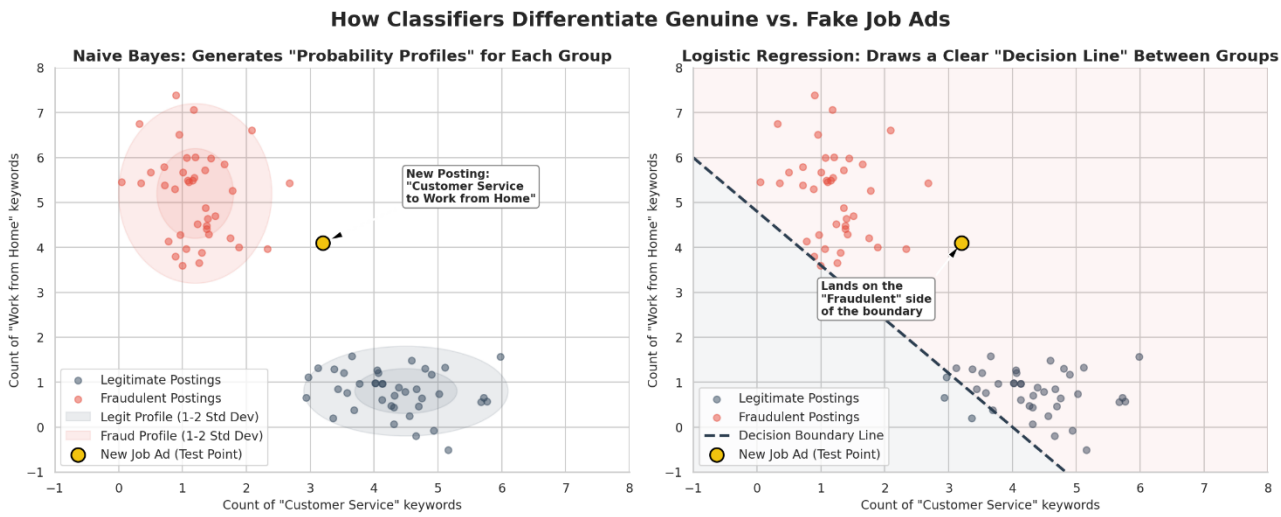
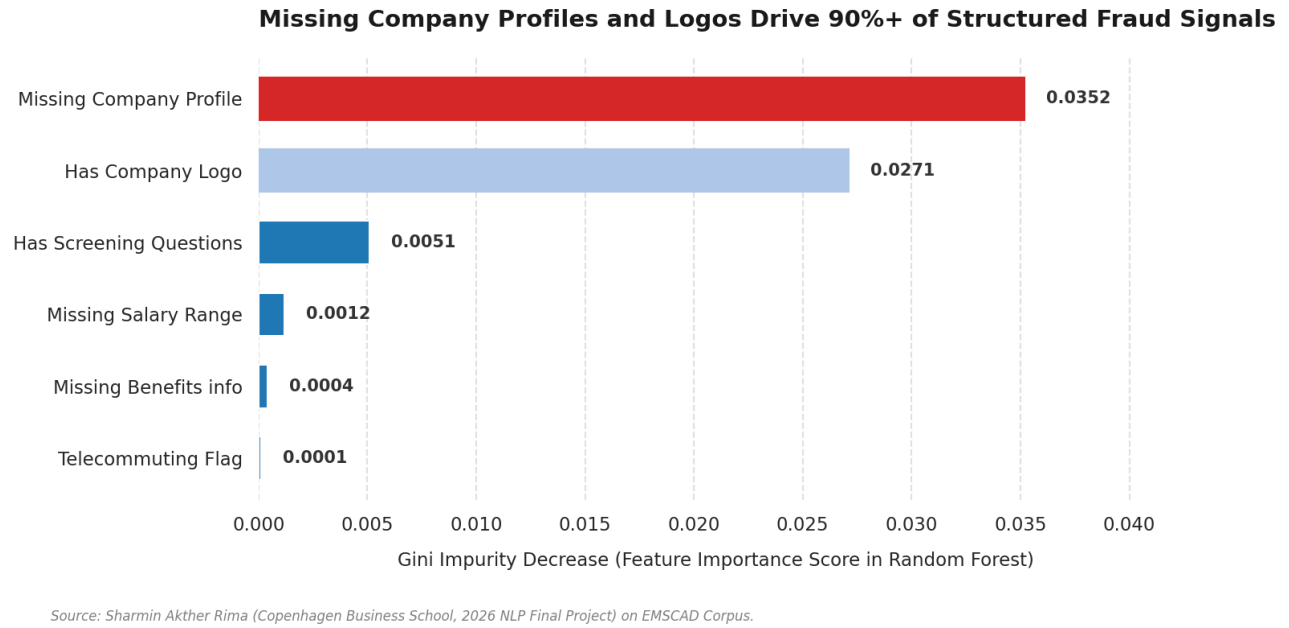


Figure 4: Baseline Tier 1 comparison of Naïve Bayes and Logistic Regression

3.6.2 Baseline Tier 2: Text + Structured Features with a Tree Model

A Random Forest is an ensemble of many decision trees, each trained on a random bootstrap sample of the data and a random subset of features at each split. Every tree "votes" on the class (Real vs Fraudulent), and the forest combines these votes (majority vote or averaged probability) into a final prediction. Because it aggregates many de-correlated trees, it reduces overfitting compared to a single decision tree and can naturally handle a mix of feature types — in Tier 2 this means it can jointly use TF-IDF text features and structured features (like missing_company_profile, has_company_logo, telecommuting) together in one model, and it also provides a built-in feature importance score based on how much each feature reduces impurity across all

trees.



Source: Sharmin Akther Rima (Copenhagen Business School, 2026 NLP Final Project) on EMSCAD Corpus.

Figure 5: Baseline Tier 2 showing combination of text scoring with structured features

3.6.3 Baseline Tier 3: Word Embeddings + Neural Network

Word2Vec is a neural embedding technique that maps each word to a dense, static 300-dimensional vector (trained via CBOW or Skip-gram) so words in similar contexts get similar vectors; each posting is then represented by averaging its word vectors into a single document vector. Cosine similarity, which compares vector directions regardless of magnitude (1.0 = identical, 0 = unrelated), is used to check whether real postings cluster more tightly than real–fraud pairs. These document vectors feed two classifiers: a Multi-layer Perceptron — a feed-forward neural network with ReLU-activated hidden layers trained via backpropagation and gradient descent — and Logistic Regression, which learns a simple linear decision boundary on the same 300-dim embeddings.

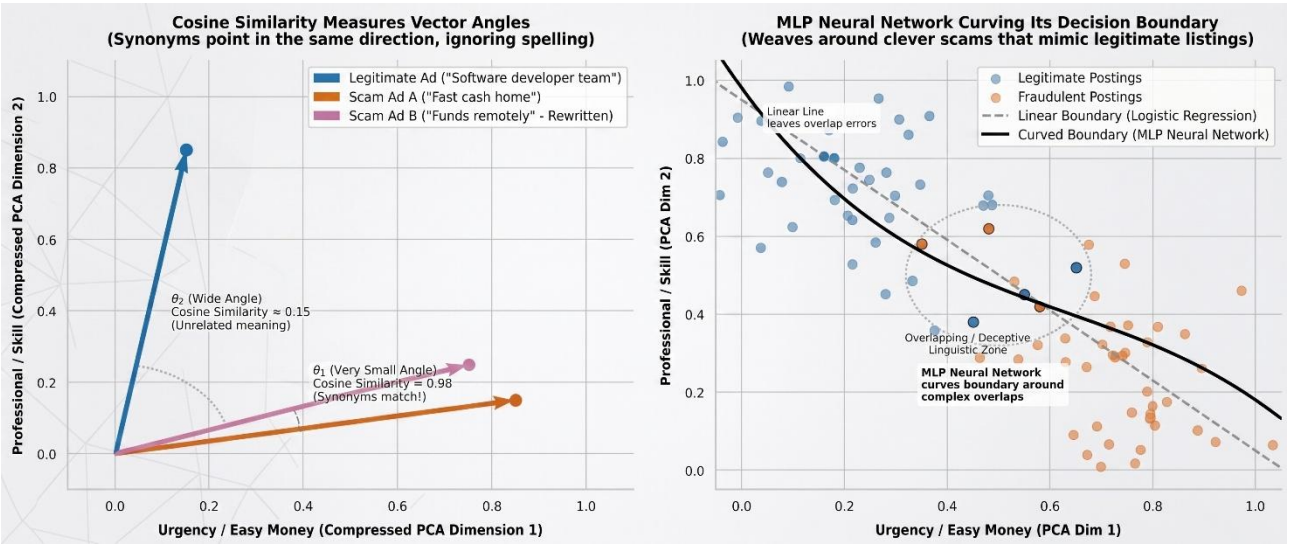


Figure 6: Baseline Tier 3 comparison of Logistic Regression and MLP Neural Network

3.6.4 Primary Model Tier 4: DistilRoBERTa (Fine-tuned Transformer)

DistilRoBERTa is a compact transformer created via knowledge distillation, where a 6-layer student is trained to imitate a 12-layer RoBERTa teacher — keeping the same hidden size and attention heads but roughly halving the layers and parameters for a faster, lighter model that retains most of the performance. It uses a Byte-Pair Encoding (BPE) tokenizer, which starts from individual characters and iteratively merges the most frequent pairs into larger chunks, so common words stay whole while rare ones are split into familiar sub-pieces. On top of the pretrained encoder, a new classification head is added and, together with light fine-tuning of the existing weights, teaches the general language model to distinguish real from fraudulent postings.

How It Works in This Pipeline

Text is tokenized into fixed-length BPE sequences (max_length=256) and wrapped as PyTorch tensors via a custom JobPostingDataset class (built to work around a bug in Hugging Face datasets). The 6-layer DistilRoBERTa encoder processes these tokens and the classification head outputs class probabilities. Class imbalance is handled with a weighted cross-entropy loss (from class_weight='balanced'), penalizing misclassification of the minority Fraudulent class more heavily. Training uses the Hugging Face Trainer with a linear warm-up scheduler, FP16 mixed precision when a GPU is available, gradient accumulation to an effective batch size of 32, and early stopping (patience 3) that tracks validation F1 and reloads the best checkpoint to curb overfitting. This tier uses a genuine three-way split: training updates weights, a held-out validation split guides early stopping and checkpoint selection, and the test set is touched only once for final evaluation.

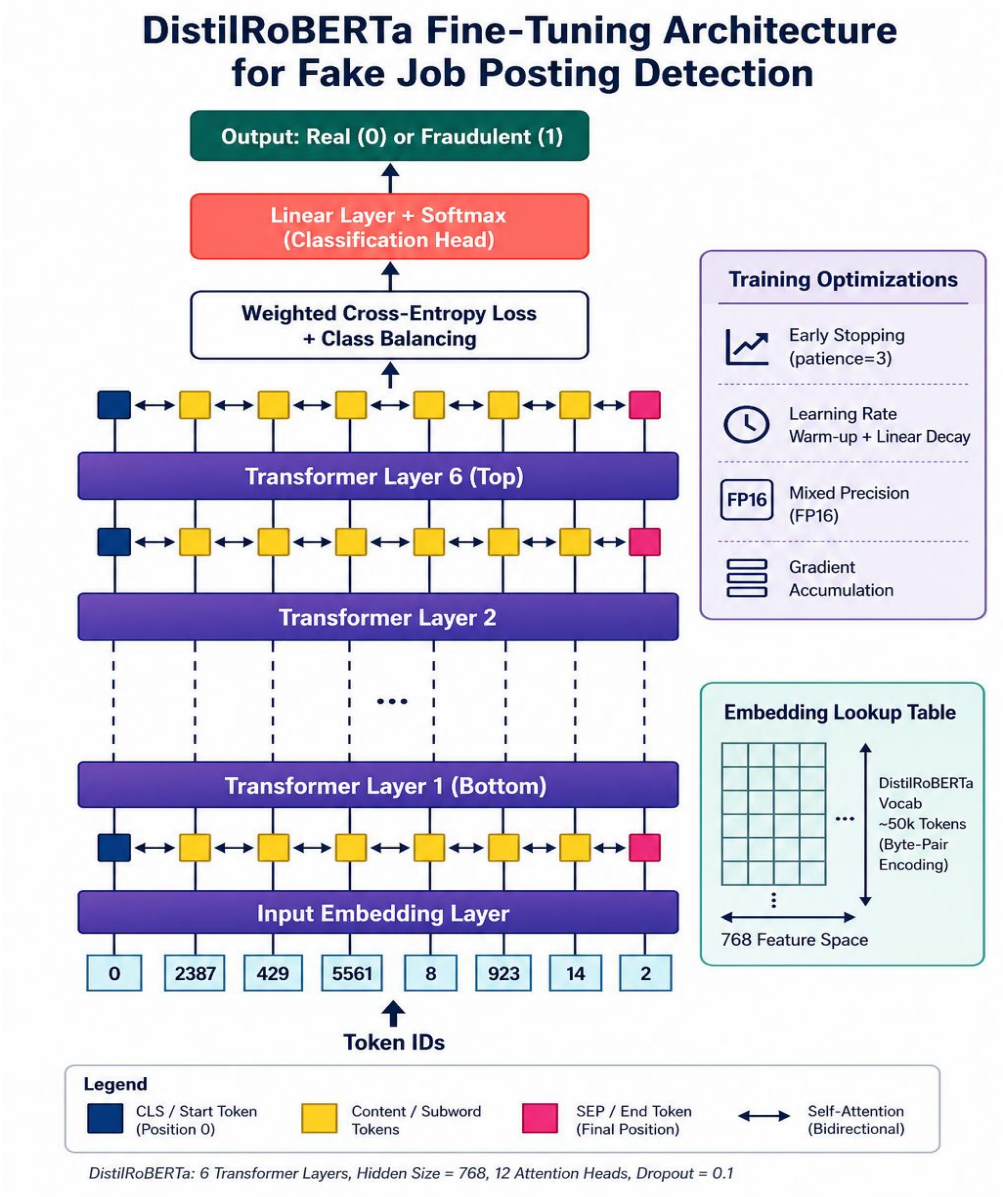


Figure 7: DistilBERTa – Fine-tuned Transformer workflows

4. Results and Discussion

4.1 Evaluation Metrics

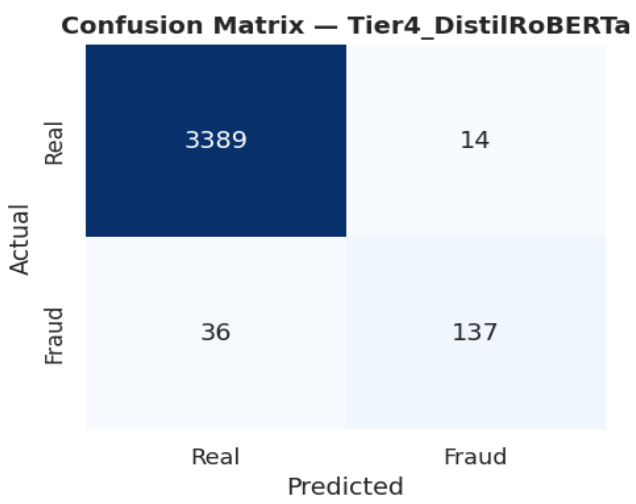
The dataset was split into 12,873 training rows, 1,431 validation rows, and 3,576 test rows, where 3403 are real, and 173 are fraudulent advertisements. Accuracy, precision, recall, and F1-score were calculated, but the F1-score of the Fraudulent class was treated as the main comparison metric. This is important because fraudulent postings represent only 4.84% of the test data, meaning that accuracy alone can give an overly positive view of performance.

4.2 Baseline models comparison

The multinomial naive Bayes model achieved 95.9% accuracy, which reflects the correct classification of real job ads. But this performed badly on the minority class, missed 116 fraudulent postings, proving it is unsuitable for detecting scams. TF-IDF with logistic regression performed better with the highest recall of 0.873, and fraud F1-score of 0.766, correctly identifying 151 of 173 fraudulent postings. These results indicate the superiority of a discriminative linear boundary over probabilistic independence. Random forest model, achieving 98% accuracy and fraud F1 of 0.733, produced no false-positive predictions at all (precision = 1.00) but missed 73 fraud cases. The feature-importance graph showed that missing_company_profile was the most influential structured feature, followed by others. This supports the EDA finding that missing or incomplete company information is associated with fraudulent postings. However, feature importance indicates how frequently a feature contributes to tree decisions; it does not establish that the feature causes fraud. Word2Vec with MLP beats Word2Vec with logistic regression on producing false positives (MLP having 26, while LR having 477) and a large difference in fraud F1 score (0.787 for MLP and 0.363 for LR). These give more importance to a non-linear classifier to extract more useful decision patterns than a linear model.

4.3 Primary Model – DistilBERTa

DistilRoBERTa produced the best overall result, achieved 98.6% accuracy, fraud precision of 0.907, recall of 0.792 and F1-score of 0.846. Its confusion matrix contained 3,389 true negatives, 14 false positives, 36 false negatives and 137 true positives. It therefore produced the strongest balance between detecting fraudulent advertisements and avoiding incorrect fraud warnings.



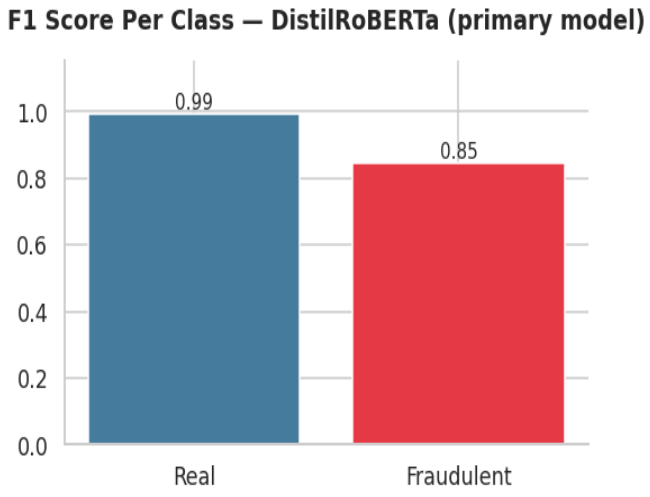


Figure 8 & 9: Confusion matrix and F1 score distribution by class for DistilBERTa

The validation F1-score improved from 0.734 in epoch 1 to its highest value of 0.828 in epoch 5, before decreasing slightly to 0.819 in epoch 6. Although the training loss continued to decrease, the validation loss started increasing after epoch 3. This suggests that the model began to overfit by learning the training data too closely. Therefore, the checkpoint with the highest validation F1-score was selected. F1-score was more suitable than validation loss because the dataset was imbalanced and F1 better reflects how well the model detected fraudulent postings.

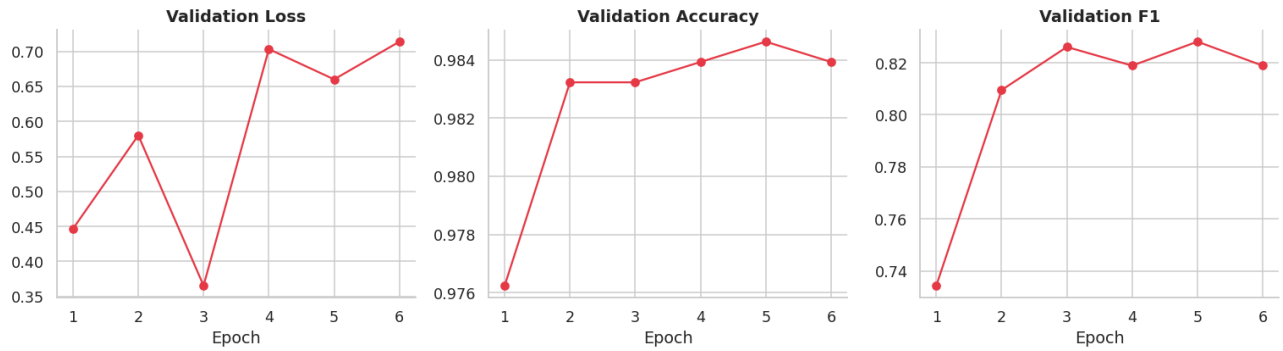


Figure 10: Validation loss, Validation accuracy and Validation F1

The PCA (Principal Component Analysis) projection indicates that the fraudulent advertising activities and the real ones are not separated by clusters. Averaging also removes word order and can dilute important phrases within long postings. So more complex models such as neural networks and DistilBERTa are required to capture hidden patterns, and understand the context of the sentences so that scammers' used phrases and tricks today are identified.

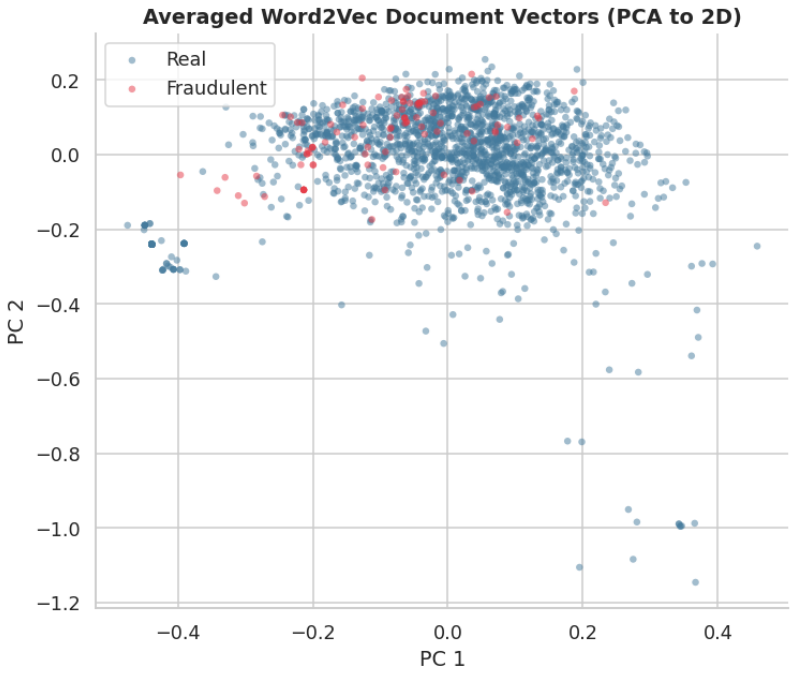


Figure 11: PCA Visualization

Overall, DistilRoBERTa was the strongest model because it achieved the highest fraud F1-score while maintaining high precision and recall. However, TF-IDF Logistic Regression provided the highest recall and represents a practical, computationally efficient alternative for human-reviewed screening. The findings

suggest that contextual transformers offer the best overall balance, while simpler models remain useful when interpretability, speed or maximum scam detection is prioritised.

Model	Accuracy	Fraud precision	Fraud recall	Fraud F1	False positives	False negatives
TF-IDF + Naive Bayes	0.959	0.648	0.329	0.437	31	116
TF-IDF + Logistic Regression	0.974	0.683	**0.873**	0.766	70	**22**
TF-IDF + structured features + Random Forest	0.980	**1.000**	0.578	0.733	**0**	73
Word2Vec + Logistic Regression	0.859	0.232	0.832	0.363	477	29
Word2Vec + MLP	0.980	0.832	0.746	0.787	26	44
DistilRoBERTa	**0.986**	0.907	0.792	**0.846**	14	36

Table 2: Tabular comparison among Baseline Models and Proposed Model

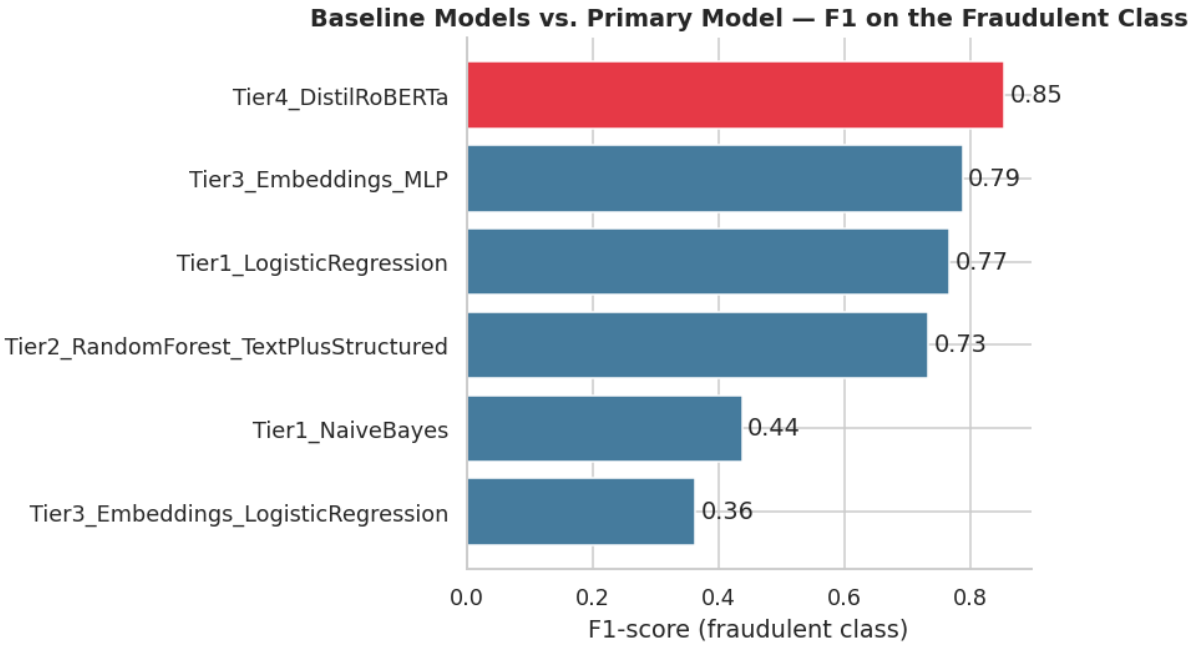


Figure 12: Comparison among Baseline Models and the Proposed Model on F1 score

5. Limitations

5.1 Dataset-related limitations:

EMSCAD dataset containing data from over a decade ago (2012-2014), the predicted model on this data does not reflect newer scam language patterns and tactics, such as AI- generated job ads or postings that attempt to redirect victims to informal communication methods like Telegram or Whatsapp. The dataset was limited to

only English-written postings, not considering the fraudulent ads in other languages. This also leads to a large number of fake posts on multilingual platforms.

5.2 Class imbalance handling:

Extreme class imbalance required using class_weight='balanced' to handle low-frequency class (fake jobs, only 4.8% of the data). In this method, extra mathematical weight is given to the scams during training instead of using SMOTE, which creates nonsensical synthetic text. Though data escaped from becoming messy and confusing, it led the models to be restricted to learning from only 866 real scams. This limited the models' efficiency in detecting a variety of deceptive behaviors.

5.3 Unreliable feature limitation:

EDA generated the single most useful structured feature - missing company profile - was a usual among scammers a decade ago, and cannot be applied to today's advanced posting formats. In the era of technology, it is very easy for a skilled scammer to bypass the system by putting more effort and knowledge into designing a fake company profile. Moreover, other missing features prove that they are skipped by real companies, also leaving no room for differentiation.

5.4 Model weaknesses:

The Naive Bayes model ignores sentence context as it depends on sparse vectors and treats text strictly as a bag-of-words. Also, it is limited by probability estimation bias, giving more emphasis on the majority class when faced with imbalance. Random forest showed weakness in missing 42% of the actual scams. Word2Vec lacks dynamic contextual embeddings. The primary model, DistilBERTa, is extremely limited by the free Colab GPU compute time, which forced us to use subsampling for the transformer tier instead of the full dataset. It also showed early increases in validation loss, indicating it started to simply memorize the training data rather than learning general rules. DistilRoBERTa is also limited to reading 512 tokens at a time. Because its tokenizer breaks complex words into multiple pieces, a normal 400-word posting easily hits this limit, forcing the system to permanently leave the bottom half of the text. The current LLM was also tested on a sample size of 60 job postings because of high API costs, and as a result, it was a failure because there was a minimum of 1 fake job ad per sample. Furthermore, GPT zero-shot prompting without proper instruction tuning made it impossible to compare its performance fairly against the other tiers that processed the whole test set.

5.5 Evaluation Limitations:

The models were evaluated on a single stratified 80/20 split rather than k-fold cross-validation, looking at only 173 test set scams. As a result, sampling variance exists in the F1/precision /recall results, and final scores are just point-in-time estimates. To prove one model as the best one, deeper statistical testing is required.

5.6 Accuracy Rate Confusion

The accuracy for the DistilRoBERTa model, the 98.46% validation accuracy and the 98.6% final test accuracy differ because they represent scores on completely different batches of data: the validation set is used as a progress check during training, while the test set evaluates the model's final performance on untouched, unseen data. Finally, that exact 98.6% test accuracy appears as 99% in the code output simply because the Python evaluation function automatically rounds the math to two decimal places.

6. Conclusion

This project worked on a four- tier NLP pipeline for detecting fraudulent job postings on the EMSCAD dataset, where the logistic regression works better in minority class identification, having higher scores for F1(fraud) of 0.77 compared to the multinomial naive bayes model having only 0.44. Random forest is better at fraud precision (1.00), but missed over 40% of actual fraud ads. According to tier 3, the Multi-layer Perceptron captured non- linear patterns better in the embedding space with F1 of 0.79.

The fine- tuned DistilBERTa transformer, the primary model of the study, delivered the strongest results overall — 99% accuracy with a fraud-class F1 of 0.85– by leveraging contextual attention to catch professionally disguised scams that fooled simpler models, while remaining lightweight enough to train on free-tier compute. But an aging dataset that misses modern scam tactics, a 512-token truncation ceiling, GPU-driven subsampling, and reliance on a single train/test split rather than cross-validation limited the scope of this model.

Future work should extend this pipeline with a zero-shot LLM comparison, k-fold cross-validation, and periodic retraining on newer postings. Looking further ahead, job fraud detection must evolve beyond post-submission text analysis to counter automated botnets; integrating IoT telemetry and edge computing would let platforms analyze a submitter's digital footprint in real time, enabling malicious postings to be identified and blocked at the network edge before publication. This decentralized, real-time approach represents a meaningful step toward closing the vulnerability gap exploited by increasingly automated scam operations, complementing the text-based detection methods evaluated in this study.

REFERENCES:

Akram, N., Irfan, R., Al-Shamayleh, A. S., Kousar, A., Qaddos, A., Imran, M., & Akhunzada, A. (2024). Online recruitment fraud (ORF) detection using deep learning approaches. *IEEE Access, 12*, 109388–109408. https://doi.org/10.1109/ACCESS.2024.3435670

Amaar, A., Aljedaani, W., Rustam, F., Rupapara, V., Ludi, S., & Ullah, S. (2022). Detection of fake job postings by utilizing machine learning and natural language processing approaches. *Neural Processing Letters, 54*(3), 2219–2247. https://doi.org/10.1007/s11063-021-10727-z

Anita, C. S., Nagarajan, P., Sairam, G. A., Ganesh, P., & Deepakkumar, G. (2021). Fake job detection and analysis using machine learning and deep learning algorithms. *Revista Gestão Inovação e Tecnologias, 11*(2), 642–650. https://doi.org/10.47059/revistageintec.v11i2.1701

Bish, A. (2025, May 7). *Rise in reports of "shameful" recruitment scams*. BBC. https://www.bbc.com/news/articles/cg5qd3ve168o

Chorpenning, K. (2026, February 18). *30 job scams & how to protect yourself in 2026*. FlexJobs. https://www.flexjobs.com/blog/post/common-job-search-scams-how-to-protect-yourself-v2

Dutta, S., & Bandyopadhyay, S. K. (2020). Fake job recruitment detection using machine learning approach. *International Journal of Engineering Trends and Technology, 68*(4). http://www.ijettjournal.org

Federal Bureau of Investigation, Internet Crime Complaint Center. (2022, January 31). *Scammers exploit security weaknesses on job recruitment websites to impersonate legitimate businesses, threatening company reputation and defrauding job seekers*. https://www.ic3.gov/PSA/2022/PSA220201

GeeksforGeeks. (n.d.-a). *Logistic regression in machine learning*. https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/

GeeksforGeeks. (n.d.-b). *Multinomial Naive Bayes*. https://www.geeksforgeeks.org/machine-learning/multinomial-naive-bayes/

GUVI. (n.d.). *Multinomial Naive Bayes: A complete guide for text classification*. https://www.guvi.in/blog/multinomial-naive-bayes/

Hugging Face. (n.d.-a). *Byte-pair encoding tokenization*. https://huggingface.co/learn/llm-course/en/chapter6/5

IJRASET. (2026, April). *Fake job post detection using machine learning*. https://www.ijraset.com/best-journal/fake-job-post-detection-using-machine-learning

Jonathan, J., Taslim, S. V., Farrel, K., Mujhid, A., & Hidayaturrahman, H. (2025). Optimizing BERT for fake job posting detection: A comparative study for data augmentation integrity and model performance. In *Proceedings of the International Conference on Information Technology, Computer, and Communications* (pp. 263–268). https://doi.org/10.1109/ICITCOM66635.2025.11265659

Joyce, S. P. (n.d.). *Job scams: Warning signs and how to avoid them*. Job-Hunt. https://www.job-hunt.org/job-search-scams/

Kaggle. (n.d.-a). *Real/fake job posting prediction* [Data set]. https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction

Kaggle. (n.d.-b). *Recruitment scam* (Version 3) [Data set]. https://www.kaggle.com/datasets/amruthjithrajvr/recruitment-scam/versions/3

Kanimozhi, M., Gopi Chandu, K., Prudhvi Nath, G., Sharath Kumar Reddy, D., et al. (2026). Design and implementation of an AI-driven fake job posting detection system using Naïve Bayes classification and TF-IDF-based natural language processing. *International Journal of Innovative Research in Technology, 12*(12).

Kashyap, P. (n.d.). *Understanding multi-layer perceptrons and backpropagation in neural networks*. Medium. https://medium.com/@piyushkashyap045/understanding-multi-layer-perceptrons-and-backpropagation-in-neural-networks-7382d5b529d2

Kishore, G. K., Yarramsetty, E. S., Thipparthi, D., Ustalamuri, C. R., Kanchepogu, N., & Nutulapati, N. (2026). Online job scam detection using BERT embeddings and SMOTE-SMOBD. *International Journal for Research in Applied Science & Engineering Technology, 14*(4). https://doi.org/10.22214/ijraset.2026.8020

Leite, T. M. (n.d.). *Neural networks, multilayer perceptron and the backpropagation algorithm*. Medium. https://medium.com/@tiago.tmleite/neural-networks-multilayer-perceptron-and-the-backpropagation-algorithm-a5cd5b904fde

Naudé, M., Adebayo, K. J., & Nanda, R. (2023). A machine learning approach to detecting fraudulent job types. *AI & Society, 38*, 1013–1024. https://doi.org/10.1007/s00146-022-01469-0

OpenReview. (n.d.). *The SMOTE paradox: Why a 92% baseline collapsed to 6%.* https://openreview.net/pdf/e531e05f331f49c7544c3266a8b45bec521f7701.pdf

Pillai, A. S. (2023). Detecting fake job postings using bidirectional LSTM. *International Research Journal of Modernization in Engineering, Technology and Science*. https://doi.org/10.56726/IRJMETS35202

Rahman, A., & Ahmed, N. U. (2026). Enhancing online recruitment fraud detection: A comparative analysis of gradient boosting and transformer architectures under severe class imbalance. *International Journal of Computer Applications, 187*(91), 1–10. https://doi.org/10.5120/ijca2026926617

Rastogi, R. (n.d.). *Papers explained 06: DistilBERT*. Medium. https://medium.com/dair-ai/papers-explained-06-distil-bert-6f138849f871

ScienceDirect. (n.d.). *Multilayer perceptron: An overview*. https://www.sciencedirect.com/topics/computer-science/multilayer-perceptron

scikit-learn. (n.d.). *MultinomialNB*. https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html

Statistics Fundamentals. (n.d.). *Logistic regression explained: Formula, examples & uses*. https://statisticsfundamentals.com/logistic-regression/

Taneja, K., Vashishtha, J., & Ratnoo, S. (2024). Transformer based unsupervised learning approach for imbalanced text sentiment analysis of e-commerce reviews. *Procedia Computer Science, 235*, 2318–2331. https://doi.org/10.1016/j.procs.2024.04.220

Taneja, K., Vashishtha, J., & Ratnoo, S. (2025). FraudBERT: Transformer based context aware online recruitment fraud detection. *Discover Computing, 28*, Article 9. https://doi.org/10.1007/s10791-025-09502-8

Towards Data Science. (n.d.). *NLP illustrated, part 3: Word2Vec*. https://towardsdatascience.com/nlp-illustrated-part-3-word2vec-5b2e12b6a63b/

University of Portsmouth. (n.d.). *Improving imbalanced students' text feedback*. https://pure.port.ac.uk/ws/files/16305200/Improving_imbalanced_students.pdf

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)* (pp. 6000–6010). https://doi.org/10.5555/3295222.3295349

Vidros, S., Kolias, C., Kambourakis, G., & Akoglu, L. (2017). Automatic detection of online recruitment frauds: Characteristics, methods, and a public dataset. *Future Internet, 9*(1), Article 6. https://doi.org/10.3390/fi9010006

Wikipedia. (n.d.-a). *Byte-pair encoding*. Retrieved August 9, 2026, from https://en.wikipedia.org/wiki/Byte-pair_encoding

Wikipedia. (n.d.-b). *Multilayer perceptron*. Retrieved August 9, 2026, from https://en.wikipedia.org/wiki/Multilayer_perceptron

Appendix - Codes

1. Setting the environment

Part 0 — Colab Setup

```
# Installing few packages Colab doesn't ship with by default
!pip install --quiet gensim afinn transformers datasets accelerate evaluate
```

```
import torch, transformers, sklearn, nltk, gensim
print("torch:", torch.__version__)
print("transformers:", transformers.__version__)
print("scikit-learn:", sklearn.__version__)
print("GPU available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("Device:", torch.cuda.get_device_name(0))
```

```
torch: 2.11.0+cu128
transformers: 5.13.1
scikit-learn: 1.6.1
GPU available: True
Device: Tesla T4
```

2. Loading the dataset

Part 1 - Loading and Inspecting the Data

```
import pandas as pd
import numpy as np

df = pd.read_csv(DATA_PATH)
print("Shape:", df.shape)
df.info()
```

3. Output

```
Shape: (17880, 18)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17880 entries, 0 to 17879
Data columns (total 18 columns):
 #   Column               Non-Null Count  Dtype 
---  ------               --------------  ----- 
 0   job_id               17880 non-null  int64 
 1   title                17880 non-null  object
 2   location             17534 non-null  object
 3   department           6333 non-null   object
 4   salary_range         2868 non-null   object
 5   company_profile      14572 non-null  object
 6   description          17879 non-null  object
 7   requirements         15184 non-null  object
 8   benefits             10668 non-null  object
 9   telecommuting        17880 non-null  int64 
 10  has_company_logo     17880 non-null  int64 
 11  has_questions        17880 non-null  int64 
 12  employment_type      14409 non-null  object
 13  required_experience  10830 non-null  object
 14  required_education   9775 non-null   object
 15  industry             12977 non-null  object
 16  function             11425 non-null  object
 17  fraudulent           17880 non-null  int64 
dtypes: int64(5), object(13)
memory usage: 2.5+ MB
```

4. Checking for missing values

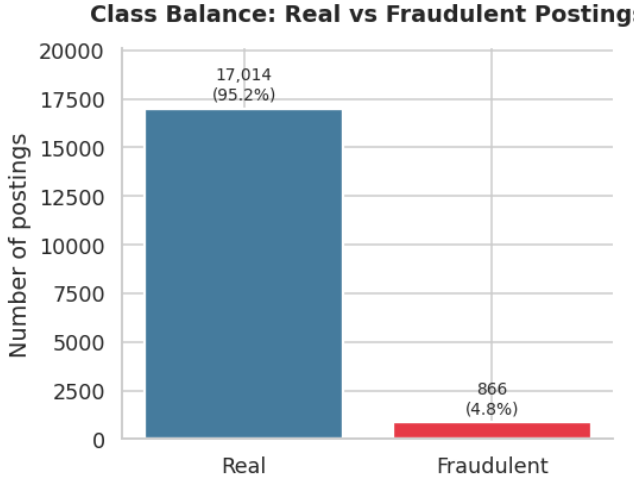
```
df.isnull().sum().sort_values(ascending=False)
```

	0
salary_range	15012
department	11547
required_education	8105
benefits	7212
required_experience	7050
function	6455
industry	4903
employment_type	3471
company_profile	3308
requirements	2696
location	346
description	1
title	0
job_id	0
telecommuting	0
has_questions	0
has_company_logo	0
fraudulent	0

dtype: int64

5. Class balance between real and fraud

```
counts = df['fraudulent'].value_counts()
pct = df['fraudulent'].value_counts(normalize=True) * 100

fig, ax = plt.subplots(figsize=(5, 4))
bars = ax.bar(['Real', 'Fraudulent'], counts.values, color=PALETTE, edgecolor='white', linewidth=1.5)
ax.bar_label(bars, labels=[f'{v:,}\n({p:.1f}%)' for v, p in zip(counts.values, pct.values)], padding=3, fontsize=9)
ax.set_ylim(0, counts.max() * 1.18)
ax.set_title('Class Balance: Real vs Fraudulent Postings', fontweight='bold', pad=15)
ax.set_ylabel('Number of postings')
sns.despine()
plt.tight_layout()
plt.show()
```



6. EDA – Missingness by Class

Part 2 — Exploratory Data Analysis

Missingness by class, text length by class, and top bigrams by class

```
cols_to_check = ['company_profile', 'benefits', 'salary_range', 'requirements', 'department']
missing_by_class = pd.DataFrame({
    col: df.groupby('fraudulent')[col].apply(lambda x: x.isnull().mean() * 100)
    for col in cols_to_check
}).T
missing_by_class.columns = ['Real (%)', 'Fraudulent (%)']
print(missing_by_class.round(1))

ax = missing_by_class.plot(kind='bar', figsize=(8, 4.5), color=PALETTE, edgecolor='white')
ax.set_title('Missingness by Class', fontweight='bold')
ax.set_ylabel('% of postings missing this field')
plt.xticks(rotation=30, ha='right')
sns.despine()
plt.tight_layout()
plt.show()
```

```
                Real (%)  Fraudulent (%)
company_profile     16.0            67.8
benefits            40.2            42.0
salary_range        84.5            74.2
requirements        14.9            17.8
department          64.7            61.3
```

7. Comparing description of fake job ads vs real

```
df['description'] = df['description'].fillna('')
df['desc_len'] = df['description'].str.split().apply(len)

fig, ax = plt.subplots(figsize=(6, 4.5))
sns.boxplot(data=df, x='fraudulent', y='desc_len', hue='fraudulent',
            palette=PALETTE, legend=False, ax=ax)
ax.set_xticks([0, 1]); ax.set_xticklabels(['Real', 'Fraudulent'])
ax.set_title('Description Length by Class', fontweight='bold')
ax.set_xlabel(''); ax.set_ylabel('Word count')
sns.despine()
plt.tight_layout()
plt.show()
print(df.groupby('fraudulent')['desc_len'].describe()[['mean', '50%', 'std']])
```

```
                mean    50%         std
fraudulent
0          171.041378  147.0  122.561633
1          158.748268  113.5  136.633010
```

8. Top bigrams to differentiate between real and fake job postings

```
from nltk import word_tokenize, ngrams
from nltk.corpus import stopwords
from collections import Counter

stop_words = set(stopwords.words('english'))

def top_bigrams(text_series, n=12):
    all_bigrams = []
    for text in text_series:
        tokens = [w.lower() for w in word_tokenize(str(text)) if w.isalpha() and w.lower() not in stop_words]
        all_bigrams.extend(ngrams(tokens, 2))
    return Counter(all_bigrams).most_common(n)

real_bg = top_bigrams(df.loc[df['fraudulent'] == 0, 'description'].sample(2000, random_state=42))
fraud_bg = top_bigrams(df.loc[df['fraudulent'] == 1, 'description'])

fig, axes = plt.subplots(1, 2, figsize=(11, 5))
for ax, data, title, color in zip(axes, [real_bg, fraud_bg], ['Real', 'Fraudulent'], PALETTE):
    labels = [' '.join(bg) for bg, _ in data][::-1]
    values = [c for _, c in data][::-1]
    ax.barh(labels, values, color=color)
    ax.set_title(f'Top Bigrams — {title}', fontweight='bold')
sns.despine()
plt.tight_layout()
plt.show()
```

9. Preprocessing and feature engineering

Part 3 — Preprocessing & Feature Engineering

```
text_cols = ['title', 'company_profile', 'description', 'requirements', 'benefits']
for c in text_cols:
    df[c] = df[c].fillna('')
df['full_text'] = df[text_cols].agg(' '.join, axis=1)
```

10. Cleaning the text

```python
# Cleaning function
import re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

lem = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    text = re.sub(r'<.*?>', ' ', text)              # strip HTML tags (postings often contain markup)
    text = re.sub(r'http\S+|www\.\S+', ' ', text)   # strip URLs
    text = re.sub(r'[^a-z\s]', ' ', text)            # keep letters only
    tokens = word_tokenize(text)
    tokens = [lem.lemmatize(t) for t in tokens if t not in stop_words and len(t) > 2]
    return ' '.join(tokens)

print(clean_text(df['full_text'].iloc[0])[:250])
```

```
marketing intern food created groundbreaking award winning cooking site support connect celebrate home cook give everything need one place top editorial business
```

+ Code + Text

```python
df['clean_text'] = df['full_text'].apply(clean_text)   # ~1-2 min for all 17,880 rows
df[['full_text', 'clean_text']].head(2)
```

	full_text	clean_text
0	Marketing Intern We're Food52, and we've creat...	marketing intern food created groundbreaking a...
1	Customer Service - Cloud Video Production 90 S...	customer service cloud video production second...

11. structured feature engineering by creating three binary missing-information flags

```python
# Structured features + engineered "missingness" flags the EDA above motivated
df['missing_company_profile'] = (df['company_profile'].str.strip() == '').astype(int)
df['missing_benefits'] = (df['benefits'].str.strip() == '').astype(int)
df['missing_salary'] = df['salary_range'].isnull().astype(int)

structured_cols = ['telecommuting', 'has_company_logo', 'has_questions',
                   'missing_company_profile', 'missing_benefits', 'missing_salary']
df[structured_cols].head()
```

	telecommuting	has_company_logo	has_questions	missing_company_profile	missing_benefits	missing_salary
0	0	1	0	0	1	1
1	0	1	0	0	0	1
2	0	1	0	0	1	1
3	0	1	0	0	0	1
4	0	1	1	0	0	1

12. Fraud rate calculation in train and test sets

```python
from sklearn.model_selection import train_test_split

X_text = df['clean_text']
X_struct = df[structured_cols]
y = df['fraudulent']

X_text_train, X_text_test, X_struct_train, X_struct_test, y_train, y_test = train_test_split(
    X_text, X_struct, y, test_size=0.2, stratify=y, random_state=42
)
print("Train:", X_text_train.shape[0], " Test:", X_text_test.shape[0])
print("Fraud rate — train:", round(y_train.mean(), 4), " test:", round(y_test.mean(), 4))
```

```
Train: 14304  Test: 3576
Fraud rate — train: 0.0484  test: 0.0484
```

13. Baseline Model 1

Part 4 — Baseline Tier 1: Bag-of-Words (Naive Bayes + Logistic Regression)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression

tfidf = TfidfVectorizer(ngram_range=(1, 2), max_features=10000)
X_train_tfidf = tfidf.fit_transform(X_text_train)
X_test_tfidf = tfidf.transform(X_text_test)   # transform only on test — never re-fit (Lab 6 note)

nb = MultinomialNB()   # alpha=1.0 by default = Laplace smoothing, the standard fix for
                       # the "zero frequency problem" (an unseen word would otherwise get P=0)
nb.fit(X_train_tfidf, y_train)
nb_preds = nb.predict(X_test_tfidf)
evaluate('Tier1_NaiveBayes', y_test, nb_preds)
```

```
--- Tier1_NaiveBayes ---
              precision    recall  f1-score   support

        Real       0.97      0.99      0.98      3403
  Fraudulent       0.65      0.33      0.44       173

    accuracy                           0.96      3576
   macro avg       0.81      0.66      0.71      3576
weighted avg       0.95      0.96      0.95      3576
```

```
logreg = LogisticRegression(max_iter=1000, class_weight='balanced')
logreg.fit(X_train_tfidf, y_train)
logreg_preds = logreg.predict(X_test_tfidf)
evaluate('Tier1_LogisticRegression', y_test, logreg_preds)
```

```
--- Tier1_LogisticRegression ---
              precision    recall  f1-score   support

        Real       0.99      0.98      0.99      3403
  Fraudulent       0.68      0.87      0.77       173

    accuracy                           0.97      3576
   macro avg       0.84      0.93      0.88      3576
weighted avg       0.98      0.97      0.98      3576
```

14. Baseline Model 2

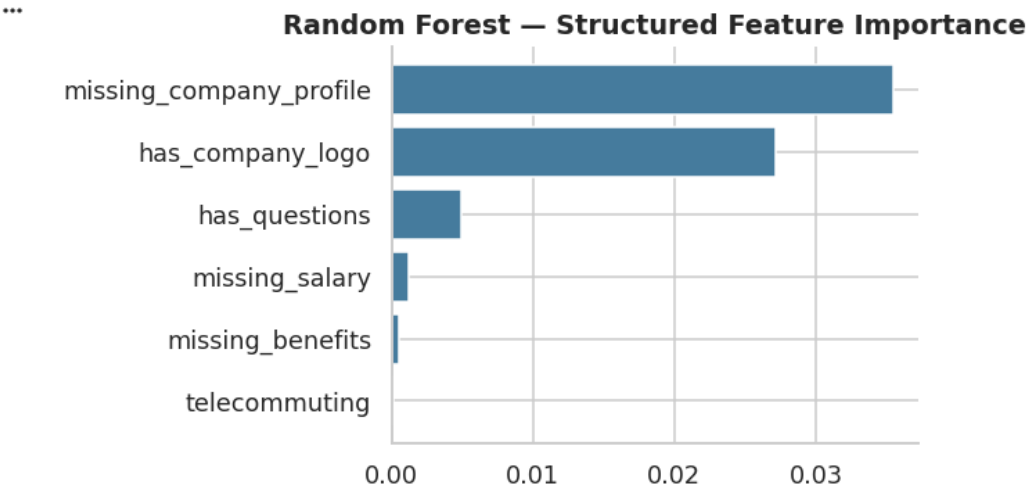
Part 5 — Baseline Tier 2: Text + Structured Features with a Tree Model

Combines the TF-IDF matrix with the structured features from Part 3 and feeds both into a Random Forest — the "tree model"

```
from scipy.sparse import hstack
from sklearn.ensemble import RandomForestClassifier

X_train_combined = hstack([X_train_tfidf, X_struct_train.values])
X_test_combined = hstack([X_test_tfidf, X_struct_test.values])

rf = RandomForestClassifier(n_estimators=200, class_weight='balanced', random_state=42, n_jobs=-1)
rf.fit(X_train_combined, y_train)
rf_preds = rf.predict(X_test_combined)
evaluate('Tier2_RandomForest_TextPlusStructured', y_test, rf_preds)
```

```
--- Tier2_RandomForest_TextPlusStructured ---
              precision    recall  f1-score   support

        Real       0.98      1.00      0.99      3403
  Fraudulent       1.00      0.58      0.73       173

    accuracy                           0.98      3576
   macro avg       0.99      0.79      0.86      3576
weighted avg       0.98      0.98      0.98      3576
```

```
n_text_features = X_train_tfidf.shape[1]
struct_importances = rf.feature_importances_[n_text_features:]

fig, ax = plt.subplots(figsize=(6, 3.5))
order = np.argsort(struct_importances)
ax.barh(np.array(structured_cols)[order], struct_importances[order], color=COLOR_REAL)
ax.set_title('Random Forest — Structured Feature Importance', fontweight='bold')
sns.despine()
plt.tight_layout()
plt.show()
```



15. Baseline Model 3

Part 6 — Baseline Tier 3: Word Embeddings + Neural Network

```
import gensim.downloader as api
wv = api.load('word2vec-google-news-300')   # ~1.6GB, downloads once
NUM_FEATURES = 300
```

```
from nltk.tokenize import word_tokenize

def avg_feature_vector(words, model, num_features=NUM_FEATURES):
    feature_vec = np.zeros((num_features,), dtype='float32')
    n_words = 0
    for word in words:
        if word in model:
            feature_vec = np.add(feature_vec, model[word])
            n_words += 1
    return feature_vec / n_words if n_words > 0 else feature_vec

X_train_tok = X_text_train.apply(word_tokenize)
X_test_tok = X_text_test.apply(word_tokenize)
X_train_emb = np.vstack(X_train_tok.apply(lambda t: avg_feature_vector(t, wv)))
X_test_emb = np.vstack(X_test_tok.apply(lambda t: avg_feature_vector(t, wv)))
print(X_train_emb.shape, X_test_emb.shape)
```

```
(14304, 300) (3576, 300)
```

16. Visualizing PCA

```
# Visualizing document vectors: PCA down to 2D, coloured by class (Word2Vec is a *static*,
# non-contextual embedding — the same word gets the same vector everywhere it appears; contrast
# this with the *contextual* embeddings BERT produces in Part 7).
from sklearn.decomposition import PCA

pca = PCA(n_components=2, random_state=42)
emb_2d = pca.fit_transform(X_train_emb[:2000])   # subsample for a readable plot
labels_2d = y_train.values[:2000]

fig, ax = plt.subplots(figsize=(7, 6))
for label, color, name in [(0, COLOR_REAL, 'Real'), (1, COLOR_FRAUD, 'Fraudulent')]:
    mask = labels_2d == label
    ax.scatter(emb_2d[mask, 0], emb_2d[mask, 1], c=color, label=name, alpha=0.5, s=18, edgecolor='none')
ax.set_title('Averaged Word2Vec Document Vectors (PCA to 2D)', fontweight='bold')
ax.set_xlabel('PC 1'); ax.set_ylabel('PC 2')
ax.legend()
sns.despine()
plt.tight_layout()
plt.show()
```

17. Cosine Similarity measurement

```
# Cosine similarity: are two real postings more alike than a real and a fraud posting?
from sklearn.metrics.pairwise import cosine_similarity

idx_real = y_train[y_train == 0].index[:2]
idx_fraud = y_train[y_train == 1].index[:1]

vec_real1 = avg_feature_vector(word_tokenize(X_text_train.loc[idx_real[0]]), wv).reshape(1, -1)
vec_real2 = avg_feature_vector(word_tokenize(X_text_train.loc[idx_real[1]]), wv).reshape(1, -1)
vec_fraud = avg_feature_vector(word_tokenize(X_text_train.loc[idx_fraud[0]]), wv).reshape(1, -1)

print("cosine(real, real)  =", cosine_similarity(vec_real1, vec_real2)[0][0].round(3))
print("cosine(real, fraud) =", cosine_similarity(vec_real1, vec_fraud)[0][0].round(3))
```

```
cosine(real, real)  = 0.884
cosine(real, fraud) = 0.776
```

18. Embeddings with Logistic Regression

```
from sklearn.linear_model import LogisticRegression

logreg_emb = LogisticRegression(max_iter=1000, class_weight='balanced')
logreg_emb.fit(X_train_emb, y_train)
logreg_emb_preds = logreg_emb.predict(X_test_emb)
evaluate('Tier3_Embeddings_LogisticRegression', y_test, logreg_emb_preds)
```

```
--- Tier3_Embeddings_LogisticRegression ---
              precision    recall  f1-score   support

        Real       0.99      0.86      0.92      3403
  Fraudulent       0.23      0.83      0.36       173

    accuracy                           0.86      3576
   macro avg       0.61      0.85      0.64      3576
weighted avg       0.95      0.86      0.89      3576
```

19. Embeddings with MLP

```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(hidden_layer_sizes=(50, 100, 50), max_iter=300, random_state=42)
mlp.fit(X_train_emb, y_train)
mlp_preds = mlp.predict(X_test_emb)
evaluate('Tier3_Embeddings_MLP', y_test, mlp_preds)
```

```
--- Tier3_Embeddings_MLP ---
              precision    recall  f1-score   support

        Real       0.99      0.99      0.99      3403
  Fraudulent       0.83      0.75      0.79       173

    accuracy                           0.98      3576
   macro avg       0.91      0.87      0.89      3576
weighted avg       0.98      0.98      0.98      3576
```

20. Primary Model – Fine-tuning transfer model

Part 7 — PRIMARY MODEL (Tier 4): Transformer Fine-Tuning

```
# 7.1 Data for the primary model
# Tier 4 is the primary model, so by default it trains on the FULL training set (the same
# 14,304 rows Tiers 1-3 used) — more data is the single biggest lever for transformer
# performance. Only switch this to False if you are demonstrably short on GPU time.
USE_FULL_TRAINING_SET = True
QUICK_MODE_SAMPLE_SIZE = 4000   # only used if USE_FULL_TRAINING_SET is False

if USE_FULL_TRAINING_SET:
    bert_texts_all, bert_labels_all = X_text_train.tolist(), y_train.tolist()
else:
    df_quick = X_text_train.to_frame('clean_text').join(y_train)
    df_quick = df_quick.groupby('fraudulent', group_keys=False).apply(
        lambda x: x.sample(int(QUICK_MODE_SAMPLE_SIZE * len(x) / len(df_quick)), random_state=42)
    )
    bert_texts_all, bert_labels_all = df_quick['clean_text'].tolist(), df_quick['fraudulent'].tolist()

# Carve a VALIDATION split out of the training data (for early stopping / best-checkpoint
# selection) — kept separate from the final test set, which is only touched once, at the end.
bert_train_texts, bert_val_texts, bert_train_labels, bert_val_labels = train_test_split(
    bert_texts_all, bert_labels_all, test_size=0.1, stratify=bert_labels_all, random_state=42
)
print(f"Training rows:   {len(bert_train_texts)}")
print(f"Validation rows: {len(bert_val_texts)}  (used for early stopping only)")
print(f"Test rows:       {len(X_text_test)}  (untouched until final evaluation, Part 7.8)")
```

```
Training rows:   12873
Validation rows: 1431  (used for early stopping only)
Test rows:       3576  (untouched until final evaluation, Part 7.8)
```

21. Tokenization and data format conversion

```
# 7.2 Tokenization and data format conversion — Byte-Pair Encoding + PyTorch tensors
from transformers import AutoTokenizer
from datasets import Dataset
import datasets
datasets.config.TORCHVISION_AVAILABLE = False

MODEL_NAME = 'distilroberta-base'
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize_function(batch):
    return tokenizer(batch['text'], padding='max_length', truncation=True, max_length=256)

train_ds = Dataset.from_pandas(pd.DataFrame({'text': bert_train_texts, 'labels': bert_train_labels}))
val_ds = Dataset.from_pandas(pd.DataFrame({'text': bert_val_texts, 'labels': bert_val_labels}))

train_ds = train_ds.map(tokenize_function, batched=True)
val_ds = val_ds.map(tokenize_function, batched=True)

# Converts the columns directly into PyTorch tensors so they can be fed straight into the model
train_ds.set_format('torch', columns=['input_ids', 'attention_mask', 'labels'])
val_ds.set_format('torch', columns=['input_ids', 'attention_mask', 'labels'])
```

22. Model Construction

```python
# 7.3 Model construction — encoder + classification head, with label mapping
from transformers import AutoModelForSequenceClassification

id_to_label = {0: 'Real', 1: 'Fraudulent'}
label_to_id = {v: k for k, v in id_to_label.items()}

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=2, id2label=id_to_label, label2id=label_to_id,
    attn_implementation='eager'
)
# from_pretrained loads the pretrained DistilRoBERTa *encoder* and automatically appends a
# fresh, randomly-initialised *classification head* sized to num_labels — everything up to the
# head is pretrained; only the head + fine-tuning adapt it to fraud/real.
```

```
model.safetensors: reconstructing file: 100% 331MB / 331MB, 30.8MB/s
model.safetensors: downloading bytes: 314MB, 28.9MB/s
Loading weights: 100% 101/101 [00:00<00:00, 3414.79it/s]
[transformers] RobertaForSequenceClassification LOAD REPORT from: distilroberta-base
Key                          | Status     |
-----------------------------+------------+-
lm_head.layer_norm.weight    | UNEXPECTED |
lm_head.dense.bias           | UNEXPECTED |
lm_head.layer_norm.bias      | UNEXPECTED |
roberta.pooler.dense.bias    | UNEXPECTED |
lm_head.bias                 | UNEXPECTED |
lm_head.dense.weight         | UNEXPECTED |
roberta.pooler.dense.weight  | UNEXPECTED |
classifier.dense.weight      | MISSING    |
classifier.dense.bias        | MISSING    |
classifier.out_proj.bias     | MISSING    |
classifier.out_proj.weight   | MISSING    |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

```python
import torch
# 7.4 Weighted cross-entropy

from sklearn.utils.class_weight import compute_class_weight
from torch import nn
from transformers import Trainer, TrainingArguments, EarlyStoppingCallback
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

class_weights = compute_class_weight(class_weight='balanced', classes=np.array([0, 1]), y=np.array(bert_train_labels))
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
class_weights_t = torch.tensor(class_weights, dtype=torch.float).to(device)
print("Computed class weights:", class_weights)

class WeightedTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop('labels')
        outputs = model(**inputs)
        loss = nn.CrossEntropyLoss(weight=class_weights_t)(outputs.logits, labels)
        return (loss, outputs) if return_outputs else loss

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='binary', pos_label=1)
    return {'accuracy': accuracy_score(labels, preds), 'precision': precision, 'recall': recall, 'f1': f1}
```

```
Computed class weights: [ 0.52547147 10.31490385]
```

```python
# 7.5 Training optimization — learning rate + scheduler, AMP, gradient accumulation, early stopping
USE_FP16 = torch.cuda.is_available()   # Automatic Mixed Precision only works on GPU

training_args = TrainingArguments(
    # If you mounted Google Drive earlier, consider pointing this at a Drive path instead —
    # e.g. '/content/drive/MyDrive/bert_results' — so a disconnect doesn't lose your checkpoints.
    output_dir='./bert_results',
    num_train_epochs=6,                 # generous ceiling; early stopping will cut this short
    learning_rate=3e-5,                 # standard fine-tuning LR for transformer-based models
    lr_scheduler_type='linear',         # linear decay after a warm-up phase
    warmup_ratio=0.1,                   # first 10% of steps ramp the LR up from 0
    per_device_train_batch_size=16,
    gradient_accumulation_steps=2,      # effective batch size = 16 x 2 = 32, without needing more GPU memory
    per_device_eval_batch_size=32,
    fp16=USE_FP16,                      # Automatic Mixed Precision — faster, less memory, GPU only
    eval_strategy='epoch',
    save_strategy='epoch',
    save_total_limit=1,
    load_best_model_at_end=True,        # "Best Model Reloading"
    metric_for_best_model='f1',
    greater_is_better=True,
    logging_steps=50,
    report_to='none',
)
```

```python
trainer = WeightedTrainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=val_ds,                # validation split — NOT the test set — drives early stopping
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=3)],  # stop after 3 epochs with no F1 improvement
)


trainer.train()
```

```
[transformers] warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
Attempting to fix ImportError by reinstalling torch and torchvision for CUDA 13.0...
```

[2418/2418 12:47, Epoch 6/6]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.825937	0.446769	0.976240	0.796610	0.681159	0.734375
2	0.586730	0.580161	0.983229	0.894737	0.739130	0.809524
3	0.510549	0.365428	0.983229	0.826087	0.826087	0.826087
4	0.155695	0.703362	0.983927	0.896552	0.753623	0.818898
5	0.062857	0.660120	0.984626	0.898305	0.768116	0.828125
6	0.076359	0.714003	0.983927	0.896552	0.753623	0.818898

```
Writing model shards: 100% 1/1 [00:00<00:00,  1.08it/s]
Writing model shards: 100% 1/1 [00:09<00:00,  9.36s/it]
Writing model shards: 100% 1/1 [00:07<00:00,  7.21s/it]
Writing model shards: 100% 1/1 [00:01<00:00,  1.76s/it]
Writing model shards: 100% 1/1 [00:11<00:00, 11.92s/it]
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
TrainOutput(global_step=2418, training_loss=0.4588546948160782, metrics={'train_runtime': 768.6391, 'train_samples_per_second': 100.487, 'train_steps_per_second': 3.146, 'total_flos': 5115758468696064.0, 'train_loss':
0.4588546948160782, 'epoch': 6.0})
```

```python
# 7.6 Training curve visualization — validation loss, accuracy, F1 across epochs
log = pd.DataFrame(trainer.state.log_history)
eval_log = log[log['eval_loss'].notna()] if 'eval_loss' in log.columns else log

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, col, title in zip(axes, ['eval_loss', 'eval_accuracy', 'eval_f1'],
                          ['Validation Loss', 'Validation Accuracy', 'Validation F1']):
    if col in eval_log.columns:
        ax.plot(eval_log['epoch'], eval_log[col], marker='o', color=COLOR_FRAUD)
    ax.set_title(title, fontweight='bold')
    ax.set_xlabel('Epoch')
sns.despine()
plt.tight_layout()
plt.show()
```

23. Final Evaluation

7.7 — Final evaluation on the untouched test set

```
# 7.7 Final test-set evaluation (inference only — no training happens here)
test_ds = Dataset.from_pandas(pd.DataFrame({'text': X_text_test.tolist(), 'labels': y_test.tolist()}))
test_ds = test_ds.map(tokenize_function, batched=True)
test_ds.set_format('torch', columns=['input_ids', 'attention_mask', 'labels'])

preds_output = trainer.predict(test_ds)
bert_preds = np.argmax(preds_output.predictions, axis=1)
bert_report = evaluate('Tier4_DistilRoBERTa', y_test, bert_preds)   # this key feeds the master table
```

```
--- Tier4_DistilRoBERTa ---
              precision    recall  f1-score   support

        Real       0.99      1.00      0.99      3403
  Fraudulent       0.91      0.79      0.85       173

    accuracy                           0.99      3576
   macro avg       0.95      0.89      0.92      3576
weighted avg       0.99      0.99      0.99      3576
```

24. F1 score per class

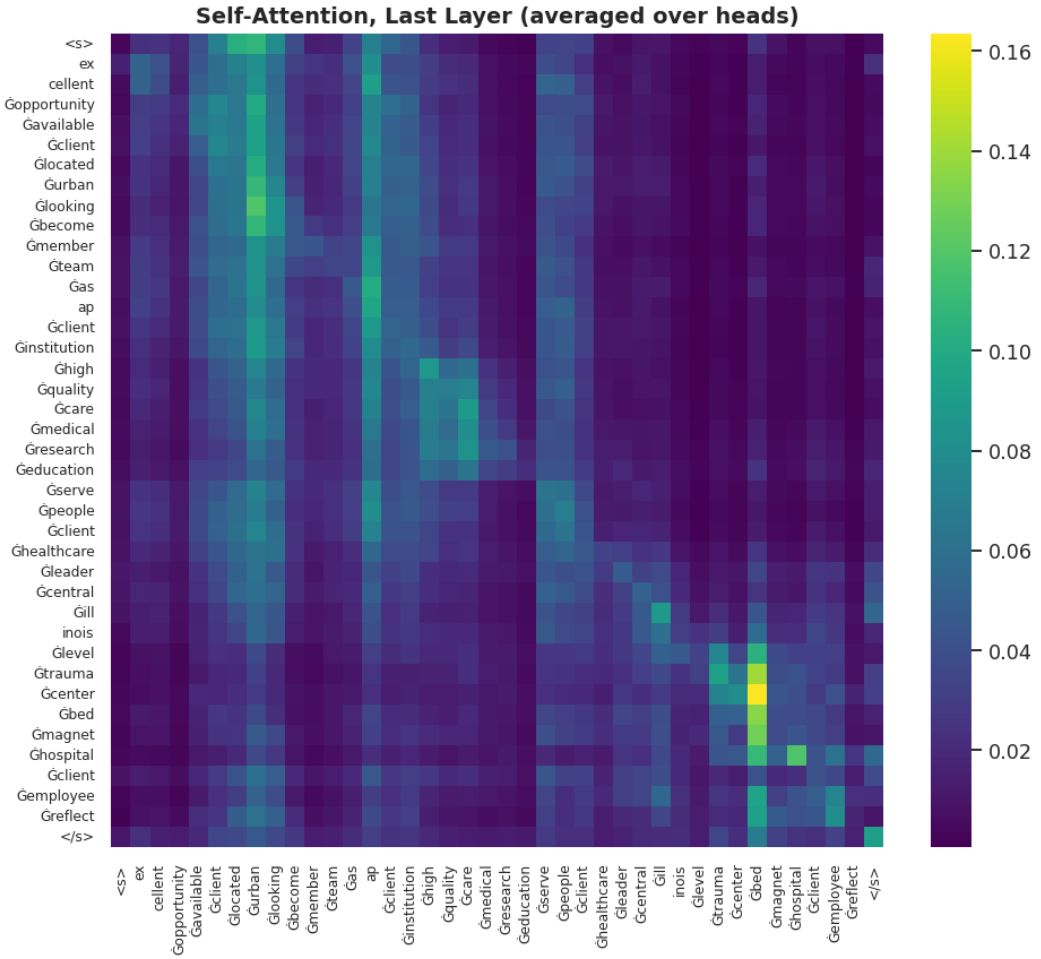
```
# F1 scores per class
per_class_f1 = {'Real': bert_report['Real']['f1-score'], 'Fraudulent': bert_report['Fraudulent']['f1-score']}
fig, ax = plt.subplots(figsize=(5, 3.5))
bars = ax.bar(per_class_f1.keys(), per_class_f1.values(), color=PALETTE)
ax.bar_label(bars, fmt='%.2f', fontsize=10)
ax.set_ylim(0, 1.15)
ax.set_title('F1 Score Per Class — DistilRoBERTa (primary model)', fontweight='bold', pad=15)
sns.despine()
plt.tight_layout()
plt.show()
```

25. Comparison among all models

```
# Baseline models (Tiers 1-3) vs. Primary Model — swap in whichever baseline tier
# actually won. Both rows come from the identical 3,576-row test set.
comparison = pd.DataFrame({
    'Tier1_NaiveBayes (baseline)': results['Tier1_NaiveBayes'],
    'Tier1_LogisticRegression (baseline)': results['Tier1_LogisticRegression'],
    'Tier2_RandomForest_TextPlusStructured (baseline)': results['Tier2_RandomForest_TextPlusStructured'],
    'Tier3_Embeddings_LogisticRegression (baseline)': results['Tier3_Embeddings_LogisticRegression'],
    'Tier4_DistilRoBERTa (primary model)': results['Tier4_DistilRoBERTa'],
}).T[['accuracy', 'precision_fraud', 'recall_fraud', 'f1_fraud']]

ax = comparison.plot(kind='bar', figsize=(12, 4.5), color=sns.color_palette('Blues_d', 4))
ax.set_title('Baseline Models vs. Primary Model (same test set)', fontweight='bold')
ax.set_ylabel('Score')
plt.xticks(rotation=25, ha='right', fontsize=9)
sns.despine()
plt.tight_layout()
plt.show()
```

26. Attention Visualization

```python
# Attention visualization for a single posting
sample_text = X_text_test.iloc[0]
inputs = tokenizer(sample_text, return_tensors='pt', truncation=True, max_length=40).to(device)
model.eval()
with torch.no_grad():
    model.config.output_attentions = True # Ensure attentions are output
    outputs = model(**inputs, output_attentions=True)

# outputs.attentions: one tensor per layer, shape (batch, num_heads, seq_len, seq_len).
# Average over heads in the LAST layer — the layer closest to the classification decision.
last_layer_attn = outputs.attentions[-1][0].mean(dim=0).cpu().numpy()
tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])

fig, ax = plt.subplots(figsize=(9, 8))
sns.heatmap(last_layer_attn, xticklabels=tokens, yticklabels=tokens, cmap='viridis', ax=ax, cbar=True)
ax.set_title('Self-Attention, Last Layer (averaged over heads)', fontweight='bold')
plt.xticks(rotation=90, fontsize=8); plt.yticks(fontsize=8)
plt.tight_layout()
plt.show()
```



27. Consolidated Comparison

Part 9 — Consolidate Results: Baseline Models vs. Primary Model

```
results_df = pd.DataFrame(results).T[['accuracy', 'precision_fraud', 'recall_fraud', 'f1_fraud']]
results_df = results_df.sort_values('f1_fraud', ascending=False)

# Label each row by its role in the project, not just its tier number — makes the table
# self-explanatory in your report without needing the tier-numbering scheme explained again.
role_map = {
    'Tier1_NaiveBayes': 'Baseline', 'Tier1_LogisticRegression': 'Baseline',
    'Tier2_RandomForest_TextPlusStructured': 'Baseline',
    'Tier3_Embeddings_LogisticRegression': 'Baseline', 'Tier3_Embeddings_MLP': 'Baseline',
    'Tier4_DistilRoBERTa': 'PRIMARY MODEL',
}
results_df.insert(0, 'Role', [role_map.get(i, 'Comparison') for i in results_df.index])
print(results_df.round(3))

fig, ax = plt.subplots(figsize=(8, 5))
bar_colors = [COLOR_FRAUD if r == 'PRIMARY MODEL' else COLOR_REAL for r in results_df['Role']]
bars = ax.barh(results_df.index, results_df['f1_fraud'], color=bar_colors)
ax.bar_label(bars, fmt='%.2f', padding=3)
ax.set_xlabel('F1-score (fraudulent class)')
ax.set_title('Baseline Models vs. Primary Model — F1 on the Fraudulent Class', fontweight='bold')
ax.invert_yaxis()
sns.despine()
plt.tight_layout()
plt.show()
```

```
                                         Role  accuracy  \
Tier4_DistilRoBERTa                PRIMARY MODEL     0.986
Tier3_Embeddings_MLP                    Baseline     0.980
Tier1_LogisticRegression                Baseline     0.974
Tier2_RandomForest_TextPlusStructured   Baseline     0.980
Tier1_NaiveBayes                        Baseline     0.959
Tier3_Embeddings_LogisticRegression     Baseline     0.859

                                   precision_fraud  recall_fraud  f1_fraud
Tier4_DistilRoBERTa                          0.907         0.792     0.846
Tier3_Embeddings_MLP                         0.832         0.746     0.787
Tier1_LogisticRegression                     0.683         0.873     0.766
Tier2_RandomForest_TextPlusStructured        1.000         0.578     0.733
Tier1_NaiveBayes                             0.648         0.329     0.437
Tier3_Embeddings_LogisticRegression          0.232         0.832     0.363
```